

AI – COMPUTER VISION

--pip install opencv-python ultralytics

1) **Real-Time Camera + Face Detection**

Concepts: OpenCV, webcam, grayscale, Haar Cascade, face detection.

```
import cv2

# Load the pre-trained face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml"
)

# Open laptop camera
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        print("Camera not found")
        break

    # Convert image to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5
    )

    # Draw rectangles around faces
    for x, y, w, h in faces:
        cv2.rectangle(
            frame,
            (x, y),
            (x + w, y + h),
            (0, 255, 0),
            2
        )

    cv2.putText(
        frame,
        "Face",
        (x, y - 10),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 255, 0),
```

```

        2
    )

    cv2.imshow("Face Detection", frame)

    # Press Q to quit
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

    cap.release()
    cv2.destroyAllWindows()

```

2) Beginner–Intermediate — Color Object Detection

Concepts: HSV, image masking, contours, object detection. This program detects a **colored object** using the webcam.

```

import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Convert BGR image to HSV
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # RED color range
    lower_red = np.array([0, 120, 70])
    upper_red = np.array([10, 255, 255])

    # Create mask
    mask = cv2.inRange(hsv, lower_red, upper_red)

    # Find contours
    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    for contour in contours:

        area = cv2.contourArea(contour)

        if area > 500:

```

```

x, y, w, h = cv2.boundingRect(contour)

# Draw bounding box
cv2.rectangle(
    frame,
    (x, y),
    (x + w, y + h),
    (0, 255, 0),
    2
)

cv2.putText(
    frame,
    "Red Object",
    (x, y - 10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 255, 0),
    2
)

cv2.imshow("Color Object Detection", frame)
cv2.imshow("Mask", mask)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()

```

3) Intermediate — YOLO Object Detection

Concepts: AI-based computer vision, YOLO, object detection, bounding boxes, confidence.
 --pip install ultralytics

```

from ultralytics import YOLO
import cv2

# Load YOLO model
model = YOLO("yolo11n.pt")

# Open laptop webcam
cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        break

```

```

# Run YOLO detection
results = model(frame, verbose=False)

# Display detection results
annotated_frame = results[0].plot()

cv2.imshow(
    "YOLO Object Detection",
    annotated_frame
)

# Press Q to quit
if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()

```

YOLO can identify common objects such as:

- Person
- Car
- Bicycle
- Dog
- Cat
- Bottle
- Chair
- Laptop
- Mobile phone

4) Intermediate — YOLO Object Tracking

Concepts: object detection + tracking, object IDs, video analytics.

This is a step beyond simple detection: the system attempts to maintain an **ID for objects as they move**.

```

from ultralytics import YOLO
import cv2

```

```

# Load YOLO model
model = YOLO("yolo11n.pt")

```

```

# Open webcam
cap = cv2.VideoCapture(0)

```

```

while True:

```

```

    ret, frame = cap.read()

```

```

    if not ret:
        break

```

```

    # Run object tracking

```

```

results = model.track(
    frame,
    persist=True,
    verbose=False
)

# Draw boxes and tracking IDs
annotated_frame = results[0].plot()

cv2.imshow(
    "YOLO Object Tracking",
    annotated_frame
)

# Press Q to quit
if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()

```

You should see bounding boxes with tracking information while objects move.

TASKS:

Student task1: Modify it to display the number of faces detected.

Student task2: Change the program to detect **blue** or **green** objects.

Student task3: Modify the program to print the detected object names and confidence values.

For example:

person 0.91

laptop 0.87

bottle 0.76

Student task4: Extend the program to:

1. Count people entering the camera view.
2. Display the tracking ID.
3. Display the number of detected objects.
4. Track only **persons**.
5. Save the processed video.